

Lab 5 - How Multilingual is Multilingual BERT?

Integrated answers to questions 1-21

Anthony Saab

Rerun checklist

Most answers can be inserted directly into the report. Three checks may need code output if you want exact numbers rather than methodological answers:

- Q8: rerun the tokenizer on the exact French sentence if you want the exact mBERT WordPiece list.
- Q12: compute the exact truncation frequency for `max_length = 128`.
- Q18: print `outputs/corpus_stats.csv` if you need the full train/dev/test sentence and token table.

1. Why do we need consistent annotation?

We need consistent annotation because the experiment compares POS-tagging performance across several languages. If each language used different POS labels or different annotation rules, the results would not be comparable: a tag learned in one treebank might not mean the same thing in another. Universal Dependencies solves this by providing a shared annotation scheme, especially the shared UPOS inventory. This allows us to train and evaluate mBERT across languages using the same label space, so differences in performance can be interpreted as differences in cross-lingual transfer rather than differences in annotation conventions.

2. Why use yield rather than return? Why call list?

The function uses `yield` because it is designed as a generator. Instead of returning the whole corpus at once, it produces one sentence at a time. This is useful for `.conllu` files because a corpus can contain many sentences, and generating them progressively is more memory-efficient and easier to iterate over. A simple `return` would stop the function after producing only one value, whereas `yield` allows the function to continue and produce every sentence in the file. We call the `list` constructor when we want to materialize the generator into an actual list. This is useful when we need to inspect the full corpus, compute statistics, or reuse the data several times. So `yield` gives lazy iteration, and `list(...)` converts that lazy sequence into a concrete dataset.

3. Distribution of labels in the Sequoia test set

In the French Sequoia test set, the UPOS label distribution is strongly imbalanced. The most frequent labels are NOUN, ADP, DET, PUNCT, and VERB, while labels such as SYM, X, and SCONJ are much rarer.

UPOS	Count
NOUN	2125
ADP	1630
DET	1486
PUNCT	1084
VERB	781
ADJ	636
PROPN	480

UPOS	Count
ADV	417
PRON	398
AUX	345
NUM	257
CCONJ	227
SCONJ	106
X	38
SYM	34

From the visualization, we can infer that POS tagging is a class-imbalanced task. A model can obtain high accuracy by performing well on frequent classes such as NOUN, ADP, DET, and PUNCT, but this does not guarantee good performance on rare labels. This is why macro-F1 is important: unlike accuracy, it gives equal weight to each label and is more sensitive to poor performance on rare categories.

4. Multiword tokens in the Sequoia test set

In the Sequoia test set, the main multiword tokens are French contracted forms such as *du*, *des*, *au*, *aux*, and *desdites*. In case-sensitive form, the file also contains variants such as *Aux*, *Au*, and *DES*. They are annotated in CoNLL-U with one range line for the surface multiword token, followed by separate lines for the grammatical words that compose it. For example, *des* is represented as a multiword token line like 13-14 *des*, then split into *de* with the UPOS tag ADP and *les* with the UPOS tag DET. Similarly, *du* is split into *de* + *le*, and *au* / *aux* are split into *à* + *le* / *les*. The multiword-token range line itself does not receive a normal POS tag; the POS labels are assigned to the component words.

This matters because the surface token seen in the raw sentence does not always correspond one-to-one with the syntactic words annotated by UD. For POS tagging, we therefore have to decide whether to ignore the range line and train on the split grammatical words, or keep the surface multiword token and assign it a combined label such as ADP+DET.

5. Why did UD split multiword tokens into grammatical words?

UD splits multiword tokens into several grammatical words because the surface form does not always correspond to one syntactic unit. In French, for example, *du* is written as one token, but grammatically it corresponds to *de* + *le*, with two different functions: *de* is an adposition and *le* is a determiner. Splitting it allows UD to assign the correct POS tag, lemma, morphological features, and dependency relation to each grammatical word.

This decision is linguistically useful because it makes annotation more consistent across languages. Languages differ a lot in how they write contractions, clitics, and fused forms, so splitting multiword tokens helps represent the underlying grammatical structure rather than only the surface spelling. My opinion is that this is the right choice for linguistic annotation, especially for syntax and cross-lingual comparison. However, it creates an extra difficulty for mBERT-based POS tagging, because mBERT tokenization follows surface/subword units, while UD annotation follows grammatical words. This means that an alignment step is necessary before training.

6. Tokens with internal spaces in Sequoia

Yes, the Sequoia corpus contains tokens with internal spaces. These are mostly numerical expressions such as 50 000, 1 000, 10 000, 125 000, 1 500, and 80 000. For the mBERT POS-tagging pipeline, these spaces should be removed before tokenization, because the tokenizer expects each UD token to correspond to one input word. For example, 50 000 should be normalized to 50000. This avoids treating one annotated UD token as if it were two separate words during alignment.

7. Why does mBERT use subword tokenization?

mBERT uses subword tokenization to handle a very large multilingual vocabulary without storing every possible word form as a separate item. This is important because languages differ in morphology, spelling, compounding, and inflection. Instead of treating rare or unknown words as completely unseen, WordPiece can split them into smaller units that may already be known by the model. This reduces vocabulary size and improves generalization across languages, especially for rare words and morphologically rich languages.

8. UD tokenization vs mBERT tokenization for the French sentence

According to UD-style tokenization, the sentence is tokenized at the level of grammatical words:

```
["Pouvez", "-vous", "donner", "les", "mêmes", "garanties", "à", "le", "sein", "de",  
"l'", "Union", "Européenne"]
```

The important point is that UD treats *au* as the grammatical sequence *à + le*, and cliticized forms such as *-vous* and *l'* receive their own tokenization. By contrast, mBERT tokenizes the same sentence into WordPiece subtokens, for example:

```
["Po", "##uve", "##z", "-", "vous", "donner", "les", "mêmes", "gara", "##nties", "au",  
"sein", "de", "l", "[UNK]", "Union", "Euro", "##pée", "##nne"]
```

Note: rerun the tokenizer on the exact string before final submission if you need exact apostrophe / [UNK] behavior, because this can change depending on whether the input uses a curly apostrophe or a straight apostrophe.

9. Why is this tokenization difference a problem?

The difference is a problem because the gold POS labels are defined for UD tokens, but mBERT receives subword tokens as input. A single UD token can correspond to several mBERT subtokens, and some UD multiword tokens may be split differently from mBERT's segmentation. Therefore, the labels and the model inputs do not naturally have the same length. Before training, we must align UD word-level labels with mBERT subword-level input. Otherwise, the model would not know which POS label corresponds to which input token.

Code implementation questions

10. Function for multiword-token handling

```
def normalize_multiword_tokens(tokens, labels, mwt_expansions):
    """
    Keep UD multiword surface tokens and assign them a combined label.

    Example:
    tokens = ["Pouvez", "-vous", "donner", "au", "à", "le", "sein"]
    labels = ["VERB", "PRON", "VERB", "_", "ADP", "DET", "NOUN"]
    mwt_expansions = {"au": 2}

    Output:
    ["Pouvez", "-vous", "donner", "au", "sein"]
    ["VERB", "PRON", "VERB", "ADP+DET", "NOUN"]
    """
    new_tokens = []
    new_labels = []

    i = 0
    while i < len(tokens):
        token = tokens[i]

        if labels[i] == "_" and token in mwt_expansions:
            n_parts = mwt_expansions[token]
            component_labels = labels[i + 1:i + 1 + n_parts]

            new_tokens.append(token)
            new_labels.append(".".join(component_labels))

            i += 1 + n_parts
        else:
            new_tokens.append(token)
            new_labels.append(labels[i])
            i += 1

    return new_tokens, new_labels
```

This implements the first alignment principle: instead of decomposing a multiword token such as `au` into `à` and `le`, we keep the surface token `au` and assign it a combined label such as `ADP+DET`.

11. Function aligning labels with mBERT subtokens

```
def tokenize_and_align_labels(sentences, labels, tokenizer, max_length=128):
    """
    Tokenize UD-tokenized sentences with mBERT and align word-level POS labels
    to subword-level input.

    First subtoken of each word receives the POS label.
    Continuation subtokens, special tokens, and padding receive -100.
    """
    encoded = tokenizer(
        sentences,
        is_split_into_words=True,
        padding=True,
        truncation=True,
        max_length=max_length,
        return_tensors=None,
    )

    aligned_labels = []

    for sent_index, word_labels in enumerate(labels):
        word_ids = encoded.word_ids(batch_index=sent_index)
        previous_word_id = None
        label_ids = []

        for word_id in word_ids:
            if word_id is None:
                label_ids.append(-100)
            elif word_id != previous_word_id:
                label_ids.append(word_labels[word_id])
            else:
                label_ids.append(-100)

            previous_word_id = word_id

        aligned_labels.append(label_ids)

    encoded["labels"] = aligned_labels
    return encoded
```

This function ensures that every mBERT token has a label position. Real word-initial subtokens receive a POS label, while continuation subtokens, padding, and special tokens receive -100, which is ignored during training and evaluation.

12. Practical effect of truncation=True

The practical effect of truncation=True is that sentences longer than the maximum sequence length are cut off. This means that some subtokens are removed from the input, and the corresponding labels are removed as well. The model therefore never sees or evaluates the truncated part of the sentence.

This can bias evaluation if truncation affects some languages more than others. For example, a language that is more heavily split into subtokens will reach the maximum sequence length faster. In my experiment, Arabic has a much higher split-word rate than the European languages, so it is more exposed to possible truncation effects. To be precise, the frequency of truncation should be measured by counting how many examples exceed the maximum length before truncation.

```
def truncation_rate(sentences, tokenizer, max_length=128):
    n_total = len(sentences)
    n_truncated = 0

    for sent in sentences:
        encoded = tokenizer(
            sent,
            is_split_into_words=True,
            truncation=False,
            add_special_tokens=True,
        )
        if len(encoded["input_ids"]) > max_length:
            n_truncated += 1

    return {
```

```

        "total_sentences": n_total,
        "truncated_sentences": n_truncated,
        "truncation_rate": n_truncated / n_total if n_total else 0.0,
    }
}

```

13. Function encoding labels into integers

```

def build_label_vocab(label_sequences):
    """
    Build a stable label vocabulary from all labels in the corpus.
    """
    labels = sorted({label for sent in label_sequences for label in sent})
    label2id = {label: i for i, label in enumerate(labels)}
    id2label = {i: label for label, i in label2id.items()}
    return label2id, id2label

def encode_labels(label_sequences, label2id):
    """
    Convert string POS labels into integer label IDs.
    Keep -100 unchanged.
    """
    encoded = []

    for sent in label_sequences:
        encoded_sent = []
        for label in sent:
            if label == -100:
                encoded_sent.append(-100)
            else:
                encoded_sent.append(label2id[label])
        encoded.append(encoded_sent)

    return encoded

```

Labels must be converted into integers because the neural classifier predicts class IDs, not strings. The special value -100 is kept unchanged because it marks ignored positions.

14. Create a HuggingFace Dataset with Dataset.from_list

```

from datasets import Dataset

def create_hf_dataset(encoded_batch):
    """
    Create a HuggingFace Dataset from tokenized and aligned examples.
    """
    records = []

    for input_ids, attention_mask, labels in zip(
        encoded_batch["input_ids"],
        encoded_batch["attention_mask"],
        encoded_batch["labels"],
    ):
        records.append({
            "input_ids": input_ids,
            "attention_mask": attention_mask,
            "labels": labels,
        })

    return Dataset.from_list(records)

```

The resulting dataset contains the three keys expected by the mBERT token-classification model: input_ids, attention_mask, and labels.

15. Create train, dev, and test datasets

```
train_encoded = tokenize_and_align_labels(  
    train_sentences,  
    train_labels,  
    tokenizer,  
    max_length=128,  
)  
  
dev_encoded = tokenize_and_align_labels(  
    dev_sentences,  
    dev_labels,  
    tokenizer,  
    max_length=128,  
)  
  
test_encoded = tokenize_and_align_labels(  
    test_sentences,  
    test_labels,  
    tokenizer,  
    max_length=128,  
)  
  
train_dataset = create_hf_dataset(train_encoded)  
dev_dataset = create_hf_dataset(dev_encoded)  
test_dataset = create_hf_dataset(test_encoded)
```

This creates three separate datasets: one for training, one for validation during fine-tuning, and one for final evaluation.

16. Reporting POS-tagging accuracy during training

```
import numpy as np  
from sklearn.metrics import accuracy_score, f1_score  
  
def compute_metrics(eval_pred):  
    logits, labels = eval_pred  
    predictions = np.argmax(logits, axis=-1)  
  
    true_labels = []  
    true_predictions = []  
  
    for pred_seq, label_seq in zip(predictions, labels):  
        for pred, label in zip(pred_seq, label_seq):  
            if label != -100:  
                true_predictions.append(pred)  
                true_labels.append(label)  
  
    return {  
        "accuracy": accuracy_score(true_labels, true_predictions),  
        "macro_f1": f1_score(true_labels, true_predictions, average="macro"),  
    }  
  
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=dev_dataset,  
    compute_metrics=compute_metrics,  
)
```

Accuracy during training is important because the loss alone does not directly tell us how good the POS tagger is. Reporting accuracy and macro-F1 allows us to check whether the model is actually improving on the validation set, whether it is overfitting, and whether it performs only on frequent labels or also on rarer POS categories.

Evaluation and interpretation questions

17. Original paper languages and linguistic limitations

Pires et al. used several evaluation settings. For NER, they used CoNLL datasets for English, German, Dutch, and Spanish, plus a multilingual in-house dataset. For POS tagging, they used Universal Dependencies data for many languages, with reported comparisons including languages such as English, German, Spanish, and Italian. The linguistic problem is that some reported comparisons are not typologically balanced. A small set such as English, German, Spanish, and Italian is mostly Indo-European and mostly European. This makes it difficult to separate true multilingual transfer from effects of language family, script, word order, morphology, vocabulary overlap, and corpus size. A better design should include languages that are deliberately selected for linguistic contrast, not only for dataset availability.

18. Corpus size and possible confounds

The corpus sizes differ substantially across languages. In my training data, Arabic PADT has 6,075 training sentences, English EWT has 12,544, Spanish GSD has 14,186, French GSD has 14,450, and Italian ISDT has 13,121. Token counts also differ, from about 204k English training tokens to about 382k Spanish training tokens.

Language / Treebank	Train sentences	Train tokens	UPOS labels
Arabic PADT	6,075	223,881	17
English EWT	12,544	204,578	17
Spanish GSD	14,186	382,327	17
French GSD	14,450	354,647	16
Italian ISDT	13,121	276,014	17

This means that corpus size could confound the conclusions if the full corpora were used directly: a model trained on a larger treebank might perform better simply because it saw more annotated examples, not because the language transfers better. To make the comparison fairer, I used a reduced setup with the same number of training examples per language: 2,000 training examples, 500 validation examples, the same maximum number of steps, the same batch size, and the same maximum sequence length. However, the exact train/dev/test split sizes should still be reported, because evaluation stability also depends on the size and domain of the test set.

19. Five-language cross-lingual experiment

I trained one POS tagger for each of the five selected languages and evaluated each trained model on the test sets of all five languages. This produced a 5 x 5 transfer matrix where rows correspond to the training language and columns correspond to the test language. The five languages were Arabic PADT, English EWT, Spanish GSD, French GSD, and Italian ISDT. This choice includes one Semitic language, English, and three Romance languages, allowing the experiment to test whether cross-lingual transfer is stronger between typologically and genealogically closer languages. The strongest results are on the diagonal, meaning that same-language training and testing works best. Cross-lingual transfer is strongest among the Romance languages, especially Spanish, French, and Italian. Transfer to Arabic is weaker overall.

20. Can we use the same hyperparameters for all languages?

Yes, the same hyperparameters should be used for all languages as much as possible. This makes the comparison fairer because differences in performance are less likely to come from different training settings. In my experiment, I used the same reduced setup for each language: 2,000 training examples, 500 validation examples, 300 maximum steps, batch size 4, maximum sequence length 128, and the same mBERT checkpoint. If one language requires different hyperparameters for practical reasons, such as GPU memory problems or much longer tokenized sequences, this should be reported explicitly. Otherwise, changing hyperparameters per language could bias the comparison.

21. Analyze the result matrix

The result matrix shows that mBERT transfers POS-tagging knowledge across languages, but the transfer is not uniform. First, the diagonal scores are the highest. This means that training and testing on the same language gives the best performance, which is expected because the model sees the same treebank conventions, vocabulary distribution, morphology, and syntax during training.

Second, the Romance languages transfer very well to one another. Spanish, French, and Italian show high cross-lingual accuracy, for example Spanish -> French, Spanish -> Italian, French -> Italian, and Italian -> French. This suggests that mBERT's multilingual representations are especially useful when the source and target languages are typologically and genealogically close.

Third, English transfers reasonably well to the Romance languages, but less well to Arabic. Arabic is the weakest cross-lingual target overall. This is probably due to a combination of different script, richer morphology, heavier subword segmentation, and greater typological distance from the European languages.

Finally, macro-F1 is lower than accuracy because the POS labels are imbalanced. Frequent labels such as NOUN, ADP, DET, VERB, and PUNCT dominate accuracy, while rare labels such as INTJ, SYM, PART, and X lower macro-F1. Overall, the results support the conclusion that mBERT is multilingual, but its multilinguality is structured rather than uniform.

Integration note for the final report

These answers can be integrated either as a separate appendix or distributed into the relevant methodology, preprocessing, training, and evaluation sections of the report. The code implementation questions are grouped separately above because they correspond to concrete functions in the pipeline.